

Project 1: Gradient Descent Variants

Requirements before starting:

- 1) you must work in teams of 2 and no more than 2 (pick a team from any section feel free to find a team using a new thread on the discussion board if needed, no requirements on same section-teams) unless you have a compelling reason in which case please email me
- 2) This project requires that you have already completed all the formative assessments above.
- 3) Include the links to your video and to your code on Page 1 of your project
- 4) Include the team member names on Page 1 at the top along with the Project # and Date
- 5) Name your file starting with Course then semester, then Project Number, then both Last names of the team members, separated by underscores, such as
CSE620_Fall25_Project1_Name1_Name2

Find and download any source code for Gradient Descent and/or its variants that you can modify and run such as the Scaffold notebook below.

- **Additional examples:**
 - [Gradient descent demo notebook](#)

PROJECT SCAFFOLD NOTEBOOK

Overall goal:

In this project, you will implement and compare four optimization algorithms—Gradient Descent (GD), Newton's Method, AdaGrad, and Adam—on three 2D objective functions. You will visualize the landscapes and overlay optimization trajectories to understand how optimizer choice and hyperparameters affect convergence, speed, and solution quality (local vs. global minima, etc).

Corresponding Unit Learning Objectives for this project:

- Identify and apply suitable gradient descent techniques or their variants to solve optimization problems.

Aligned to Course Learning Objective:

1. Demonstrate theoretical and practical knowledge of classical optimization methods.
3. Write an application or adapt an existing numerical or combinatorial optimization code or tool for implementing optimization algorithms, such as the Gradient Descent, Simplex algorithm, simulated annealing, genetic algorithms, genetic niching, multi-objective optimization, etc.

Specific project tasks that will help you achieve the above learning objectives:

To achieve the above objectives, the project requires you to:

- Correctly implement or apply suitable classical optimization methods to , specifically, GD, Newton's Method, AdaGrad, and Adam.
- Tune and compare hyperparameters and their effect on convergence and robustness.
- Visualize function contours and optimization trajectories in 2D to be able to better understand how each method works under different conditions.
- Analyze why and when different optimization methods succeed or fail.

Functions To optimize (Use all three)

- 1) Convex bowl (quadratic):

$$f1(x, y) = x^2 + y^2$$

2) Rosenbrock (banana valley):

$$f2(x, y) = (1 - x)^2 + 100 (y - x^2)^2$$

3) Multimodal non-convex (cosine bumps):

$$f3(x, y) = x^2 + y^2 + 10 \cos(x) + 10 \cos(y)$$

Required Optimization methods (Experiment with all 4)

- 1) Gradient Descent (fixed learning rate α)
- 2) Newton's Method (uses Hessian inverse)
- 3) AdaGrad (per-parameter adaptive step)
- 4) Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$ by default)

What to Implement / Do:

- A. Implement or use library versions of the four optimizers (NumPy, PyTorch, JAX, etc.).
- B. For each function, pick 2–3 different **initial** points (e.g., (-2, 2), (0.5, -1.5), (3, 3)).
- C. For each optimizer, try at least 3 **step sizes** (also called learning rates) e.g., 0.001, 0.01, 0.1. Tune Newton's with damping (damping means decreasing rate gradually with each iteration) if needed.
- D. Run until convergence, i.e., $\|x_{(k+1)} - x_k\| < 1e-6$. (However, you may need to limit the number of iterations to say, 2000 in case of either very slow convergence or oscillation)
- E. Visualize the process by drawing contour plots and overlaying the optimization path.
- F. Record the number of iterations to converge, final point, and the final f value.
- G. Analyze your results by discussing the convergence behavior across functions and hyperparameters. Identify when and why methods differ.

Expected Phenomena to Observe:

- $f1$ (quadratic): Newton converges in one step from generic starts; GD converges smoothly if α is reasonable; AdaGrad/Adam also converge quickly.
- $f2$ (Rosenbrock): GD often “zigzags” or diverges unless α is very small; AdaGrad may slow down; Adam typically works well with moderate α .
- $f3$ (cosine bumps): Multiple local minima; different methods and α may land in different basins; Adam's momentum may escape shallow minima better than plain GD/AdaGrad.

Deliverables: (report, code and video):

Report: Return your answer as a well structured and organized report of no more than **8 pages + appendix for plots and tables IF too many**, with sections for each of the following:

- **Introduction Section:** stating the objective of the project and the optimization problem instances using equations.
- **Methods Section:**
 - Describing the Gradient Descent algorithm and all variants you have used in **pseudocode and equations**.
 - Describing the optimization code that you used, including its source code and simple examples of how to use it If you did not use the above notebook (and on which platform).

- **Results Section**
 - **Make a plot of each function being optimized**
 - **Make plots of each function asking with both the starting point and final solutions found**
 - **Tabulate** for each problem and **graph** the parameters values', the time to converge to a solution, number of iterations to the solution, the function values of the solutions obtained (and average and standard deviation over multiple runs).
- **Experimental Analysis Section:** An analysis of all your above results.
 - What do you observe as the problem gets more difficult and as you move from simple gradient descent to its variants?
 - How does the function's shape/surface impact the suitability of methods or parameters?
 - How does the initial solution impact your results depending on the function?
 - What about depending on the method or its parameters?
 - Compare the runtime and number of iterations to reach the converged solutions.
 - **For each function, you will test 3 initializations * 3 parameter configurations * 4 algorithms = 24 combinations.**
 - **In total, 24 * 3 functions = 72 combinations overall.**
- **Conclusion Section:** describe your overall findings based on the experiments and what you learned and could improve.
- **Appendix Section** (includes code, and any excess tabular or graph results that do not fit the maximum number of pages)

Code: file or link is acceptable.

Video: Additionally, record a brief ~5 minute video sharing your screen and following this format:

- Demo your notebook/code step-by-step, outlining your methodology and experiments (~2 minutes)
- Summarize/discuss your findings, things you thought were interesting, etc. (~2 minutes)
- Specify the most challenging/rewarding aspects of the project, and why (~1 minute)
- **It is preferred if you can provide a link to your recording rather than the file itself. A couple of examples you can try:**
 - Microsoft Sharepoint (please make the recording visible through the link you submit)
 - Google Drive (upload Mp4 to drive → share file by link)
 - YouTube (unlisted video with link)

Please submit your assignment as a compressed file containing the above (minus the video) named like **[Group member 1 last name]_[Group member 2 last name]_Assignment1.zip**, where last names are arranged alphabetically.

Grading Rubric (100 pts):

1. **Structure and completeness of the Report** with Adherence to page limit, all above required sections and all required Content as described above, and including: (80 points) divided into:
 - **Methods & hyperparameters** used (**tables & figures**) (20 points)

- **Visualization clarity & labeling** (20 pts)
 - **Trajectory plots** per function X optimizer (for the best/most interesting parameter configurations per optimizer)
 - **Quality of experiments & tuning** (20 pts)
 - **Comparison** (iterations, final loss, robustness to α and initialization)
 - **Analysis depth & insight** (20 pts)
 - **Reflections:** Which method is most robust? Which is fastest when tuned? Where do they fail? ... etc.
2. **Code organization & readability** (10 pts)
 3. **Video** - adherence to time limit and comprehensive explanation that shows you master the project methods and findings (10)